

Experiment 3: Email Spam or Ham Classification

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch:2023-2028	Due date:
Name	Ponsubash Raj R	Register No.	3122237001037

Aim

To use the word-count frequency in a email, of the most common words, to classify emails as either spam or ham.

Objective

To classify emails as spam or ham using three classification algorithms—Naive Bayes, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM)—and evaluate their performance using accuracy metrics and K-Fold cross-validation.

Libraries Used

- numpy
- pandas
- matplotlib
- scikit-learn

Theoretical Description

1. Naive Bayes Classifier

Naive Bayes is a **probabilistic classifier** based on **Bayes' Theorem**, assuming **feature independence**.

$$P(C_k | \mathbf{x}) = \frac{P(\mathbf{x} | C_k) \cdot P(C_k)}{P(\mathbf{x})}$$

$$P(\mathbf{x} \mid C_k) = \prod_{i=1}^n P(x_i \mid C_k) \Rightarrow P(C_k \mid \mathbf{x}) \propto P(C_k) \prod_{i=1}^n P(x_i \mid C_k)$$

Types:

- Gaussian NB (continuous)
- Multinomial NB (discrete counts)
- Bernoulli NB (binary features)

2. K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based algorithm. It classifies data by majority vote among the k nearest neighbors using a distance metric like Euclidean distance.

$$d(\mathbf{x}, \mathbf{x}^{(i)}) = \sqrt{\sum_{j=1}^n (x_j - x_j^{(i)})^2}$$

$$\hat{C} = \arg \max_{C_k} \sum_{\mathbf{x}^{(i)} \in \mathcal{N}_k(\mathbf{x})} \mathbb{I}(y^{(i)} = C_k)$$

There are 2 major algorithms, to improve the time complexity:

- KD-Tree
- Ball Tree

3. Support Vector Machine (SVM)

SVM is a **supervised learning algorithm** used for classification and regression. It finds an **optimal hyperplane** that maximizes the margin between different classes.

For linearly separable data, the decision boundary is:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

The optimization problem is:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad \forall i$$

Here:

- $\mathbf{w}$ is the weight vector (normal to the hyperplane).
- b is the bias term.
- Margin = $\frac{2}{\|\mathbf{w}\|}$.

Kernel Trick: For non-linearly separable data, SVM uses kernels to map input space into higher dimensions:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j)$$

Common kernels:

- Linear: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$
- Polynomial: $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j + c)^d$
- RBF (Gaussian): $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$
- Sigmoid: $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\kappa \mathbf{x}_i \cdot \mathbf{x}_j + c)$

Python Code

```

"""# IMPORT FUNCTIONS"""

import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler, MinMaxScaler, Binarizer
import time
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

"""# CONFIGURATIONS"""

DATASET_PATH = path
TRAIN_FILE = "spambase_csv.csv"

# Target variable
TARGET_COL = "class"

# Columns to drop
UNNECESSARY_COLS = []

# Categorical filling strategy
CATEGORICAL_FILL_MODE = "mode"

# Numerical fill defaults
NUMERIC_FILL_VALUES = {}

# Outlier threshold
OUTLIER_FILTER = {}

```

```

# Step 1: User selects the model type
option = int(input(
    "Choose the model:\n"
    "1. Gaussian NB\n"
    "2. Multinomial NB\n"
    "3. Bernoulli NB\n"
    "4. k-NN with KD Tree\n"
    "5. k-NN with Ball Tree\n"
    "6. k-NN varying k\n"
    "7. SVM (Linear)\n"
    "8. SVM (Polynomial)\n"
    "9. SVM (RBF)\n"
    "10. SVM (Sigmoid)\n"
))

# Step 2: Model and Param Grid Configurations
model_param_map = {
    1: {"model": GaussianNB(), "param_grid": {}},
    2: {"model": MultinomialNB(), "param_grid": {}},
    3: {"model": BernoulliNB(), "param_grid": {}},
    4: {"model": KNeighborsClassifier(algorithm="kd_tree"), "param_grid": {
        "n_neighbors": [1, 3, 5, 7]
    }},
    5: {"model": KNeighborsClassifier(algorithm="ball_tree"), "param_grid": {
        "n_neighbors": [1, 3, 5, 7]
    }},
    6: {"model": KNeighborsClassifier(), "param_grid": {
        "n_neighbors": [1, 3, 5, 7, 9],
        "algorithm": ["auto", "kd_tree", "ball_tree"]
    }},
    7: {"model": SVC(kernel="linear"), "param_grid": {
        "C": [0.1, 1, 10]
    }},
    8: {"model": SVC(kernel="poly"), "param_grid": {
        "C": [0.1, 1, 10],
        "degree": [2, 3, 4],
        "gamma": ["scale", "auto"]
    }},
    9: {"model": SVC(kernel="rbf"), "param_grid": {
        "C": [0.1, 1, 10],
        "gamma": ["scale", "auto"]
    }},
    10: {"model": SVC(kernel="sigmoid"), "param_grid": {
        "C": [0.1, 1, 10],
        "gamma": ["scale", "auto"]
    }},
}

```

```

if option not in model_param_map:
    raise ValueError("Invalid option selected!")

model_config = model_param_map[option]
base_model = model_config["model"]
param_grid = model_config["param_grid"]

"""# LOAD DATA"""

df = pd.read_csv(f"{DATASET_PATH}/{TRAIN_FILE}")
print("Shape:", df.shape)

"""# FILL MISSING VALUES"""

for col in df.columns:
    count = df[col].isna().sum()
    print(f"{col} -> {count} missing values")

"""# CHECK AND REMOVE OUTLIERS"""

corr = df.drop(columns=["class"]).corr()

# Plot heatmap
plt.figure(figsize=(24, 10))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".1f", cbar=True)
plt.title("Feature Correlation Matrix", fontsize=16)
plt.show()

"""# SPLITTING TO FEATURES AND TARGETS"""

X = df.drop(columns=[TARGET_COL])
y = df[TARGET_COL]

X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)

# Get all numerical columns except the target
features_to_scale = X_temp.select_dtypes(include=['int64', 'float64']).columns.tolist()

# Apply model-specific preprocessing
if option == 1: # GaussianNB
    scaler = StandardScaler()
    X_temp[features_to_scale] = scaler.fit_transform(X_temp[features_to_scale])

elif option in [2, 4, 5, 6, 7, 8, 9, 10]: # MNB, k-NN, SVMs
    scaler = MinMaxScaler()
    X_temp[features_to_scale] = scaler.fit_transform(X_temp[features_to_scale])

elif option == 3: # BernoulliNB

```

```

    binarizer = Binarizer(threshold=0.1)
    X_temp[features_to_scale] = binarizer.fit_transform(X_temp[features_to_scale])

else:
    raise ValueError("Invalid option for preprocessing!")

X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.5, stratify=y_temp)

"""# CROSS VALIDATION"""

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(estimator=base_model,
                    param_grid=param_grid,
                    scoring='accuracy',
                    cv=skf,
                    n_jobs=-1)
grid.fit(X_val, y_val)

print("Best hyperparameters:", grid.best_params_)

best_model = grid.best_estimator_

cv_metrics = {
    "Fold": [], "Accuracy": [], "Precision": [], "Recall": [], "F1": []
}

fold = 1
for train_idx, val_idx in skf.split(X_train, y_train):
    X_train_fold, X_val_fold = X_train.iloc[train_idx], X_train.iloc[val_idx]
    y_train_fold, y_val_fold = y_train.iloc[train_idx], y_train.iloc[val_idx]

    best_model.fit(X_train_fold, y_train_fold)
    y_pred = best_model.predict(X_val_fold)

    cv_metrics["Fold"].append(f"Fold {fold}")
    cv_metrics["Accuracy"].append(accuracy_score(y_val_fold, y_pred))
    cv_metrics["Precision"].append(precision_score(y_val_fold, y_pred, zero_division=0))
    cv_metrics["Recall"].append(recall_score(y_val_fold, y_pred))
    cv_metrics["F1"].append(f1_score(y_val_fold, y_pred))
    fold += 1

# Add average row
cv_metrics["Fold"].append("Average")
for metric in ["Accuracy", "Precision", "Recall", "F1"]:
    cv_metrics[metric].append(np.mean(cv_metrics[metric]))

print("\nCross-Validation Metrics:")

```

```

print(pd.DataFrame(cv_metrics))

""""# Validation""""

# Best model from GridSearchCV
best_model = grid.best_estimator_

# Measure training time
start_time = time.time()
best_model.fit(X_train, y_train)
training_duration = time.time() - start_time
print(f"\nModel Training Time: {training_duration:.2f} seconds\n")

def evaluate_classification(name, y_true, y_pred):
    print(f"\n{name} Metrics:")
    print(f" Accuracy : {accuracy_score(y_true, y_pred):.4f}")
    print(f" Precision: {precision_score(y_true, y_pred, zero_division=0):.4f}")
    print(f" Recall    : {recall_score(y_true, y_pred):.4f}")
    print(f" F1-Score : {f1_score(y_true, y_pred):.4f}")

    if name == "Test":
        cm = confusion_matrix(y_true, y_pred)
        plt.figure(figsize=(5, 4))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
        plt.title(f'{name} - Confusion Matrix')
        plt.xlabel('Predicted')
        plt.ylabel('Actual')
        plt.tight_layout()
        plt.show()

# Apply model-specific preprocessing
if option == 1: # GaussianNB
    X_test[features_to_scale] = scaler.transform(X_test[features_to_scale])

elif option in [2, 4, 5, 6, 7, 8, 9, 10]: # MNB, k-NN, SVMs
    X_test[features_to_scale] = scaler.transform(X_test[features_to_scale])

elif option == 3: # BernoulliNB
    X_test[features_to_scale] = binarizer.transform(X_test[features_to_scale])

for name, (X_split, y_split) in zip(["Train", "Validation", "Test"], [(X_train, y_train), (X_val, y_val)]):
    y_pred = best_model.predict(X_split)
    evaluate_classification(name, y_split, y_pred)

    if name == "Test":
        # Check if model supports probabilities
        if hasattr(best_model, "predict_proba"):
            y_probs = best_model.predict_proba(X_split)[: , 1]

```

```

elif hasattr(best_model, "decision_function"):
    y_probs = best_model.decision_function(X_split)
else:
    print("Model does not support ROC plotting.")
    continue

# Plot ROC Curve
fpr, tpr, _ = roc_curve(y_split, y_probs)
auc_score = roc_auc_score(y_split, y_probs)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f"AUC = {auc_score:.4f}")
plt.plot([0, 1], [0, 1], "k--", label="Random Guess")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend(loc="lower right")
plt.grid()
plt.tight_layout()
plt.show()

```

Performance Comparison

Table 1: Performance Comparison of Naïve Bayes Variants

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.8154	0.8876	0.8867
Precision	0.6932	0.9360	0.8869
Recall	0.9547	0.7677	0.8168
F1 Score	0.8031	0.8431	0.8503

KNN: Varying k Values

Table 2: KNN Performance for Different k Values

k	Accuracy	Precision	Recall	F1 Score
1	0.9039	0.8806	0.8747	0.8776
3	0.9011	0.8811	0.8659	0.8734
5	0.8987	0.8859	0.8532	0.8690
7	0.8965	0.8838	0.8493	0.8660

KNN: KDTree vs BallTree

Table 3: KNN Comparison: KDTree vs BallTree

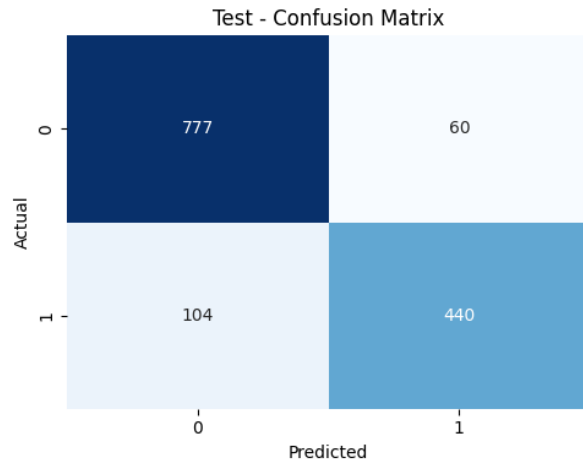
Metric	KDTree	BallTree
Accuracy	0.9039	0.9039
Precision	0.8810	0.8806
Recall	0.8742	0.8747
F1 Score	0.8775	0.8776
Training Time (s)	0.1s	0.2s

SVM Kernels

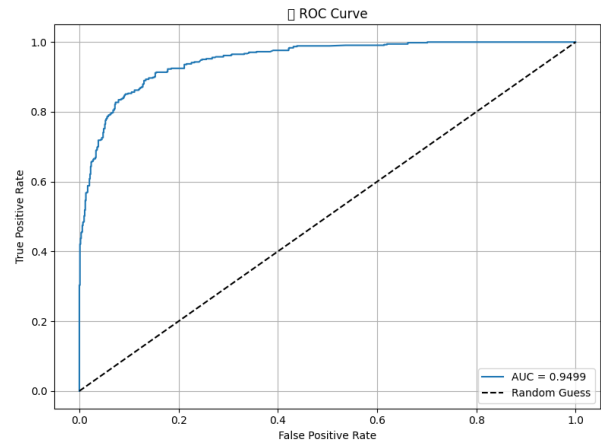
Table 4: SVM Performance with Different Kernels and Parameters

Kernel	Hyperparameters	Accuracy	F1 Score	Training Time
Linear	$C = 1$	0.9013	0.8677	0.06s
Polynomial	$C = 1$, $degree = 3$, $\gamma = 'scale'$	0.8509	0.7787	0.08s
RBF	$C = 1$, $\gamma = 'scale'$	0.9315	0.9109	0.07s
Sigmoid	$C = 1$, $\gamma = 'scale'$	0.8052	0.7482	0.12s

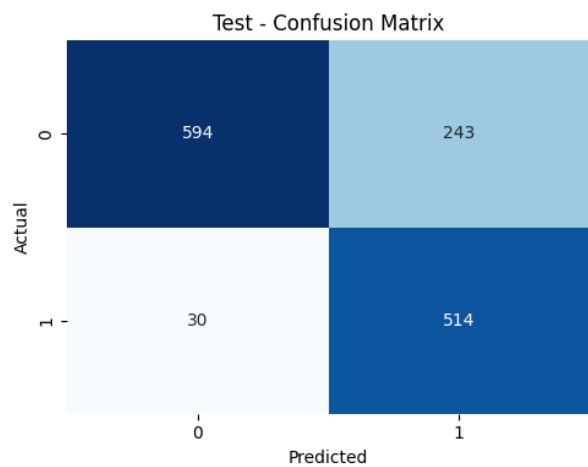
Plots



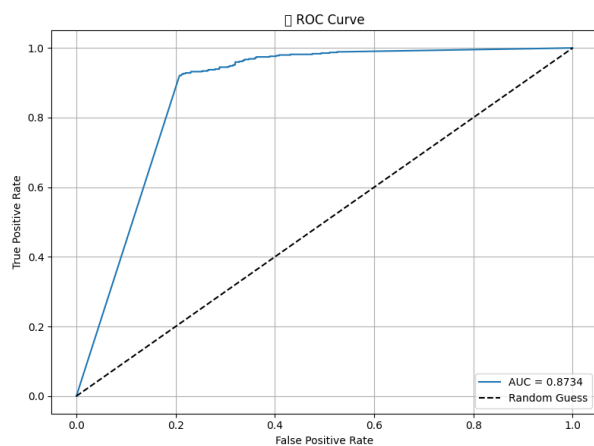
(a) Bernoulli Confusion Matrix



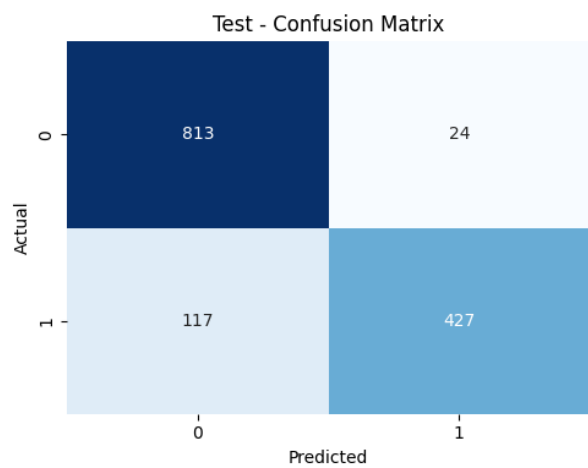
(b) Bernoulli ROC Curve



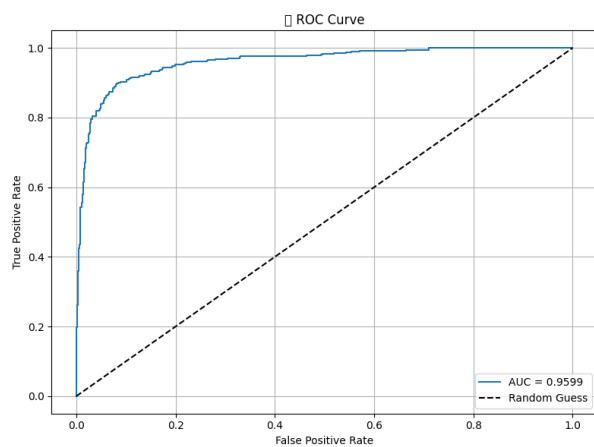
(a) Gaussian Confusion Matrix



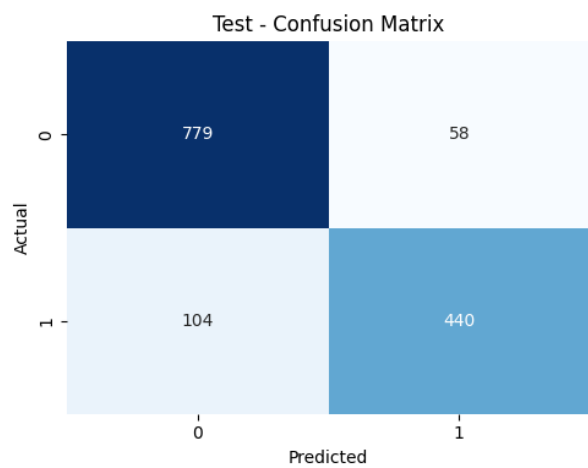
(b) Gaussian ROC Curve



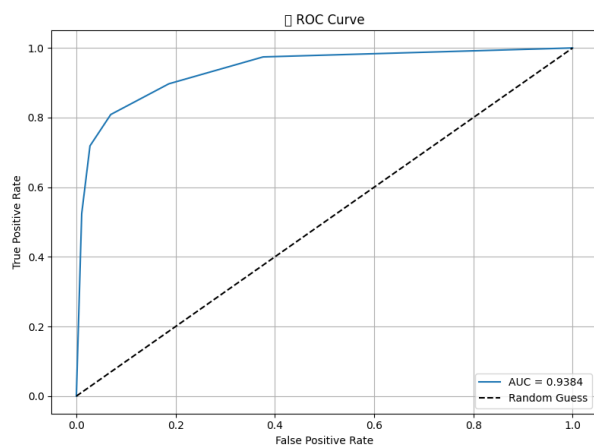
(a) Multinomial Confusion Matrix



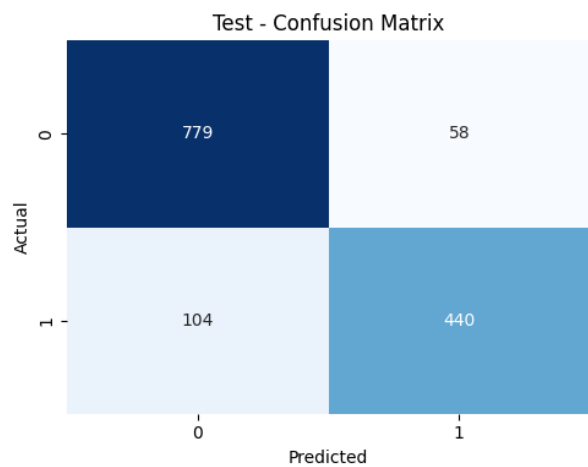
(b) Multinomial ROC Curve



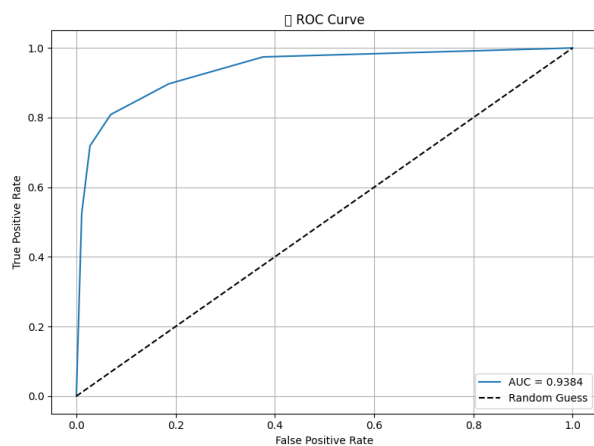
(a) KD-Tree Confusion Matrix



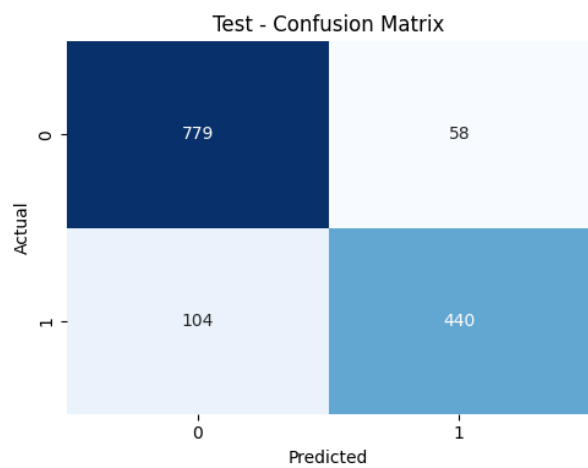
(b) KD-Tree ROC Curve



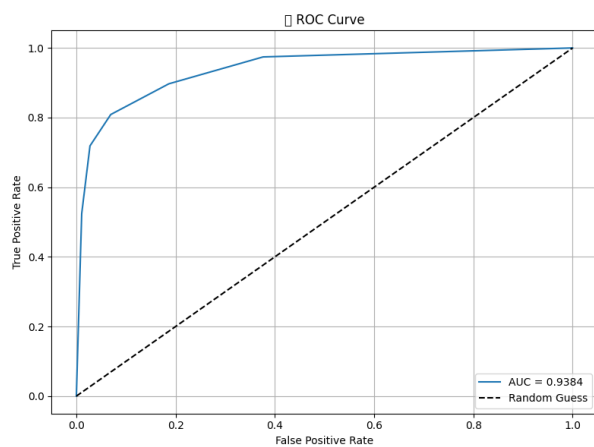
(a) Ball Tree Confusion Matrix



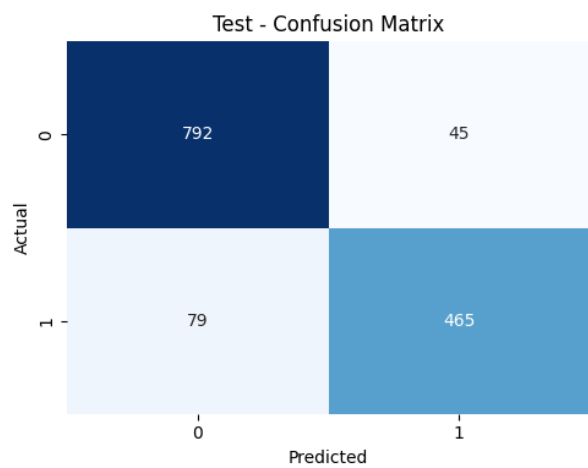
(b) Ball Tree ROC Curve



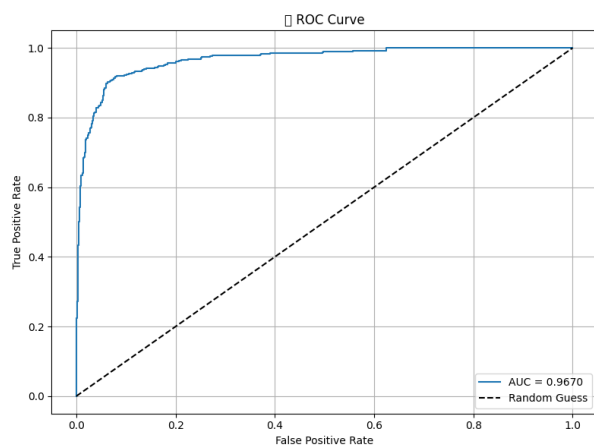
(a) KNN Confusion Matrix



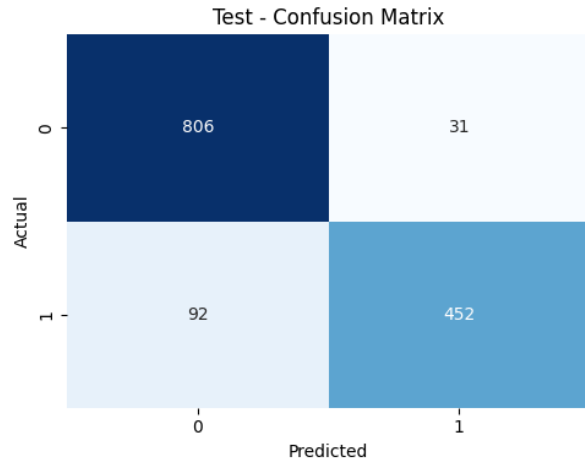
(b) KNN ROC Curve



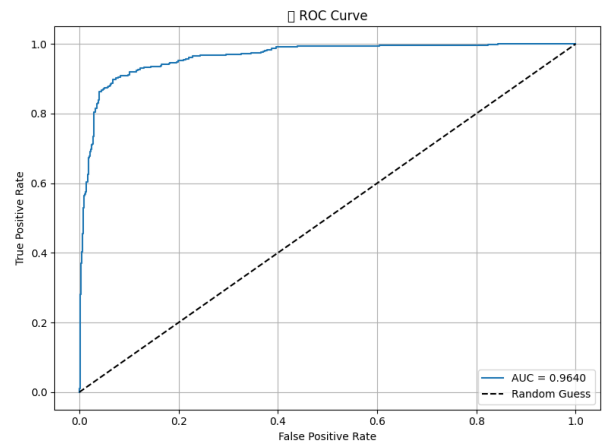
(a) SVM - Linear Kernel Confusion Matrix



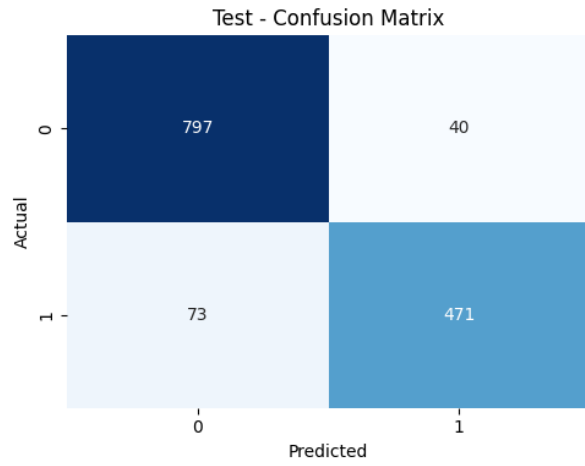
(b) SVM - Linear Kernel ROC Curve



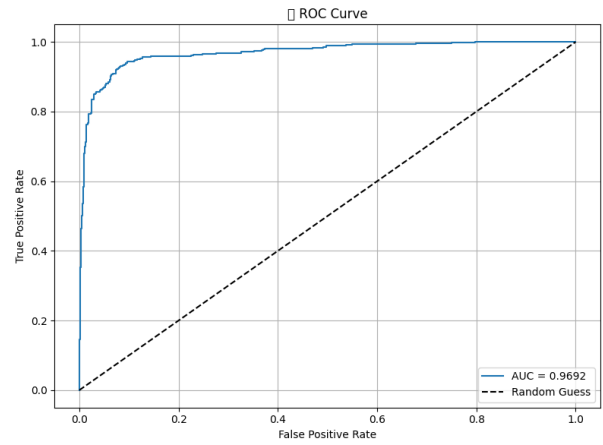
(a) SVM - Polynomial Kernel Confusion Matrix



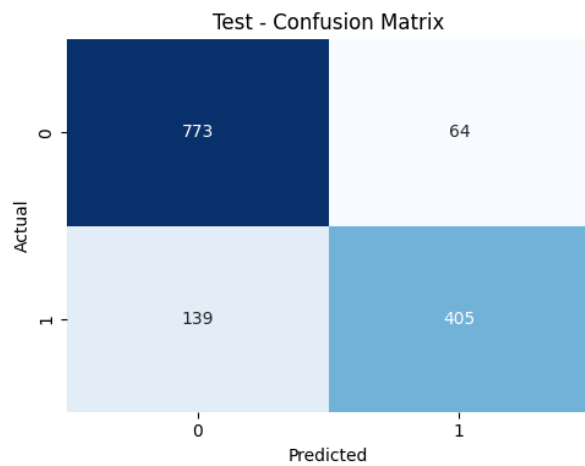
(b) SVM - Polynomial Kernel ROC Curve



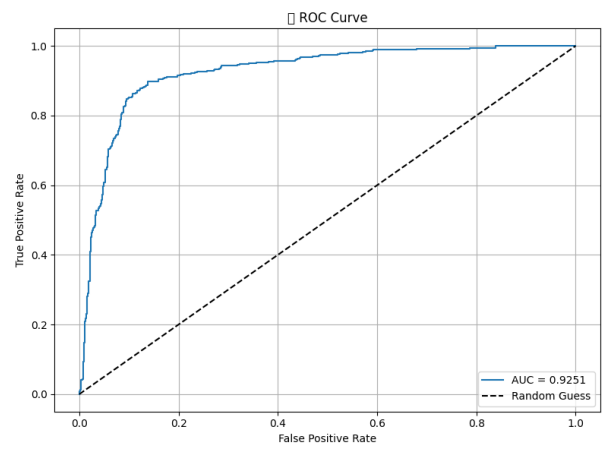
(a) SVM - RBF Kernel Confusion Matrix



(b) SVM - RBF Kernel ROC Curve



(a) SVM - Sigmoid Kernel Confusion Matrix



(b) SVM - Sigmoid Kernel ROC Curve

Across the models, the best performance is given by the SVM using RBF kernel and the worst performance is given by SVM using Sigmoid kernel.

Conclusion

- Learnt to implement supervised learning algorithms like Naive-Bayes, K-Nearest Neighbours and Support Vector Machine and its variants.
- Able to perform hyperparameter tuning and use the best set of hyper-parameters to decide the final model metrics.